- graph_v3.py — Enhanced Multi-Agent LangGraph with Dynamic Specialist Pool
  - Table of Contents
  - 1. What Does This File Do?
  - 2. Architecture Overview
    - Specialist Pool (6 agents)
  - 3. How graph_v3 Differs from graph_v2
  - 4. System-Level Architecture
  - 5. The Graph Pipeline — Visual Flow
    - Complete Graph State Machine
    - Rework Loop Detail
  - 6. State Management
    - AnalysisState (extends MessagesState)
    - State Flow Example
  - 7. The Specialist Pool
    - Fixed Stages (Always Run)
    - Dynamic Specialists (Selected by Planner)
    - Planner Decision Rules
  - 8. System Prompts — Deep Dive
    - Domain Context (Shared by ALL agents)
    - Prompt Structure Per Agent
    - Data Loader Prompt — Key Details
    - Planner Prompt — Key Details
    - Manager Review Prompt — Key Details
  - 9. Graph Builder — build_graph()
    - Function Signature
    - Build Process (Step by Step)
  - 10. Progress Reporting System
    - Stage Labels
    - Progress Messages by Node Type
      - Specialist Nodes (make_specialist_node)
      - Planner Node
      - Manager Review Node
      - Tool Node
      - Manager Router (Stage Transitions)
    - Progress Timeline Example
  - 11. Routing Logic — How Decisions Are Made
    - Three Router Functions

    - Routing Diagram
- 12. Tool Binding — Per-Specialist Tools
    - Why Tool Isolation?
- 13. Context Management — Token Budget
    - Constants
    - Context Building Strategy
    - Why Focused Context?
- 14. Manager Review & Rework Loop
- 15. File Dependencies Map
- 16. MCP Tools Reference
    - Tool Registry (7 tools)
    - execute_python Namespace
    - In-Memory DataFrame Store
- 17. Execution Examples
    - Example 1: "Analyze Mill 8 for the last 7 days"
    - Example 2: "Predict PSI80 trends for Mill 8"
    - Example 3: "Find anomalies in Mill 8 and explain root causes"
- 18. Configuration Constants
- Quick Reference: Entry Points

`graph_v3.py` — Enhanced Multi-Agent LangGraph with Dynamic Specialist Pool

> **File:** `python/agentic/graph_v3.py`
> **Role:** Planner-driven multi-agent architecture that dynamically selects specialist agents based on the user's analysis request.
> **LLM:** Google Gemini (`gemini-3.1-flash-lite-preview`)
> **Framework:** LangGraph (StateGraph) + LangChain + MCP Tools

---

Table of Contents

1. What Does This File Do?
2. Architecture Overview

3. How graph_v3 Differs from graph_v2
4. System-Level Architecture
5. The Graph Pipeline — Visual Flow
6. State Management
7. The Specialist Pool
8. System Prompts — Deep Dive
9. Graph Builder — `build_graph()`
10. Progress Reporting System
11. Routing Logic — How Decisions Are Made
12. Tool Binding — Per-Specialist Tools
13. Context Management — Token Budget
14. Manager Review & Rework Loop
15. File Dependencies Map
16. MCP Tools Reference
17. Execution Examples
18. Configuration Constants

---

1. What Does This File Do?

`graph_v3.py` is the **brain** of the agentic analysis system. It defines:

- **6 specialist agents** (analyst, forecaster, anomaly_detective, bayesian_analyst, optimizer, shift_reporter)
- A **planner agent** that reads the user's request and picks 1-4 relevant specialists
- A **manager review** gate after each specialist that can accept or request rework
- Fixed **data_loader** (start) and **code_reviewer + reporter** (end) stages
- Smart **context compression** to stay within LLM token limits

The key innovation vs graph_v2: **dynamic routing**. Instead of always running the same 4 agents, the planner selects only the relevant specialists per request.

---

2. Architecture Overview

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                          graph_v3.py — Dynamic Pipeline                      │
```

```
│                                                                                │
│   FIXED PREFIX              DYNAMIC MIDDLE              FIXED SUFFIX          │
│   ┌──────────────┐      ┌──────────────────────┐      ┌────────────────────┐   │
│   │ data_loader│      │  Selected by Planner  │      │  code_reviewer    │   │
│   │ planner    │      │  from pool of 6       │      │  reporter         │   │
│   └──────────────┘      └──────────────────────┘      └────────────────────┘   │
│                                                                                │
│   ALWAYS RUNS          1-4 SPECIALISTS              ALWAYS RUNS               │
│                         chosen per-request                                     │
└────────────────────────────────────────────────────────────────────────────────┘
```

Specialist Pool (6 agents)

```
┌──────────────────┐  ┌──────────────────┐  ┌──────────────────────┐
│    analyst       │  │   forecaster     │  │  anomaly_detective   │
│                  │  │                  │  │                      │
│  EDA, SPC,       │  │  Prophet, ARIMA, │  │  Isolation Forest,   │
│  correlations,   │  │  seasonality,    │  │  DBSCAN, SHAP,       │
│  distributions,  │  │  changepoints,   │  │  root cause,         │
│  process Cp/Cpk  │  │  trend forecast  │  │  regime detection    │
└──────────────────┘  └──────────────────┘  └──────────────────────┘

┌──────────────────┐  ┌──────────────────┐  ┌──────────────────────┐
│ bayesian_analyst │  │    optimizer     │  │   shift_reporter     │
│                  │  │                  │  │                      │
│  Bootstrap post- │  │  Pareto frontier,│  │  Per-shift KPIs,     │
│  eriors, credible│  │  what-if, Monte  │  │  benchmarking,       │
│  intervals, A/B  │  │  Carlo, setpoint │  │  energy efficiency,  │
│  testing, Cpk    │  │  recommendations │  │  handover reports    │
└──────────────────┘  └──────────────────┘  └──────────────────────┘
```

---

3. How graph_v3 Differs from graph_v2

Aspect	graph_v2	graph_v3
Agents	4 fixed (data_loader, analyst, code_reviewer, reporter)	6 specialist pool + planner + manager + fixed stages
Routing	Fixed linear pipeline	Dynamic — planner selects 1-4 specialists per request

Aspect	graph_v2	graph_v3
Specialist depth	Single generic analyst	Deep domain specialists (forecasting, anomaly, Bayesian, optimization, shifts)
Manager review	After each stage	After each stage, with rework capability
Context management	Basic	Advanced: focused context builder, compression, token budgets
Libraries	Basic (pandas, matplotlib, seaborn)	Extended: Prophet, statsmodels, pmdarima, sklearn, SHAP, hmmlearn
Recursion limit	50	150 (needed for more complex pipelines)

4. System-Level Architecture

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                          FRONTEND (Next.js)                                  │
│                                                                              │
│   /ai-chat (page.tsx)  ──►  chat-store.ts (Zustand)                         │
│        │                        │                                           │
│        │                        │  POST /api/agentic/analyze                │
│        │                        │  GET  /api/agentic/status/{id} (poll)     │
│        │                        │  GET  /api/agentic/reports/{id}/{file}    │
│        └────────────────────────┘                                           │
└──────────────────────────────────┬──────────────────────────────────────────┘
                                   │ HTTP (Next.js proxy → FastAPI :8000)
┌──────────────────────────────────┼──────────────────────────────────────────┐
│                   BACKEND        │                                          │
│                                  ▼                                          │
│   ┌──────────────────────────────────────────┐                              │
│   │  api.py (FastAPI main)                   │                              │
│   │    └── api_endpoint.py (router /api/v1/agentic)                         │
│   │          │                               │                              │
│   │          │  asyncio.create_task(         │                              │
│   │          │    _run_analysis_background()  │                              │
│   │          │  )                            │                              │
│   │          ▼                               │                              │
│   │   ┌──────────────────────────────────┐   │                              │
│   │   │  Background Task:                │   │                              │
│   │   │  1. Connect to MCP Server (:8003)│   │                              │
```

```
│  │  │  2. set_output_directory(analysis_id)   │   │                        │
│  │  │  3. get_mcp_tools(session) → client.py│   │                        │
│  │  │  4. build_graph(tools, key) → THIS FILE   │                        │
│  │  │  5. graph.ainvoke(user_question)       │   │                        │
│  │  │  6. Store result in _analyses dict     │   │                        │
│  │  └────────────────────────────────────────┘   │                        │
│  └───────────────────────────────────────────────┘                        │
│                          │                                                 │
│                          │ MCP Protocol (Streamable HTTP)                  │
│                          ▼                                                 │
│  ┌───────────────────────────────────────────────┐                        │
│  │  server.py (MCP Server, port 8003)            │                        │
│  │    │                                          │                        │
│  │    ├── tools/__init__.py (registry)           │                        │
│  │    ├── tools/db_tools.py                      │                        │
│  │    │     ├── get_db_schema                    │                        │
│  │    │     ├── query_mill_data     ──→ PostgreSQL│                        │
│  │    │     └── query_combined_data ──→ PostgreSQL│                        │
│  │    ├── tools/python_executor.py               │                        │
│  │    │     └── execute_python (pandas, Prophet,  │                        │
│  │    │         sklearn, shap, etc.)             │                        │
│  │    ├── tools/report_tools.py                  │                        │
│  │    │     ├── list_output_files                │                        │
│  │    │     └── write_markdown_report            │                        │
│  │    ├── tools/session_tools.py                 │                        │
│  │    │     └── set_output_directory             │                        │
│  │    └── tools/output_dir.py (shared state)     │                        │
│  └───────────────────────────────────────────────┘                        │
│                          │                                                 │
│                          ▼                                                 │
│  ┌───────────────────────────────────────────────┐                        │
│  │  PostgreSQL (em_pulse_data)                   │                        │
│  │    mills.MILL_01 .. MILL_12 (sensor data)     │                        │
│  │    mills.ore_quality (lab data)               │                        │
│  └───────────────────────────────────────────────┘                        │
│                                                                            │
│  ┌───────────────────────────────────────────────┐                        │
│  │  output/{analysis_id}/                        │                        │
│  │    ├── *.png (charts)                         │                        │
│  │    └── *.md  (reports)                        │                        │
│  └───────────────────────────────────────────────┘                        │
└────────────────────────────────────────────────────────────────────────────┘
```

---

5. The Graph Pipeline — Visual Flow

Complete Graph State Machine

```
                    ┌──────────────────────────────────────────────────────┐
                    │                        START                         │
```

```
                    └──────────────────────────────────────────────┘
                                   │
                                   ▼
                    ┌──────────────────────────────────────────────┐
                    │              data_loader_entry               │
                    │         (sets current_stage = "data_loader") │
                    └──────────────────────────────────────────────┘
                                   │
                                   ▼
                    ┌──────────────────────────────────────────────┐
                    │                  data_loader                 │
                    │  System Prompt: DATA_LOADER_PROMPT           │
                    │  Tools: query_mill_data, query_combined_data,│
                    │         get_db_schema                        │
                    │                                              │
                    │  Job: Load requested mill data from PostgreSQL│
                    │  Output: DataFrames stored in in-memory store│
                    └──────────────────────────────────────────────┘
                                 │                 │
                                 │ has tool_calls  │ no tool_calls
                                 ▼                 ▼
                          ┌──────────┐     ┌──────────────┐
                          │   tools  │     │ manager_review │
                          │ (execute │     │ Auto-ACCEPT for │
                          │  MCP tool)│    │ data_loader    │
                          └──────────┘     └──────────────┘
                                │                  │
                                │                  │
                                └──► data_loader   │ ACCEPT → advance
                                     (loop back)   ▼
                    ┌──────────────────────────────────────────────┐
                    │              planner_entry                   │
                    │         (sets current_stage = "planner")     │
                    └──────────────────────────────────────────────┘
                                   │
                                   ▼
                    ┌──────────────────────────────────────────────┐
                    │                   planner                    │
                    │  System Prompt: PLANNER_PROMPT               │
                    │  Tools: NONE (text-only LLM call)            │
                    │                                              │
                    │  Job: Analyze user request + loaded data summary│
                    │  Output: "SPECIALISTS: analyst, forecaster"  │
                    │          "RATIONALE: User wants trends and EDA"│
                    │                                              │
                    │  Sets state.stages_to_run = [data_loader, planner,│
                    │    analyst, forecaster, code_reviewer, reporter]│
                    └──────────────────────────────────────────────┘
                                   │
                                   ▼ (always → manager_review)
                    ┌──────────────────────────────────────────────┐
                    │              manager_review                  │
                    │  Auto-ACCEPT for planner → advance to first specialist│
                    └──────────────────────────────────────────────┘
                                   │
                                   │
          ┌────────────────────────┼────────────────────────┐
          │                        │                        │
          ▼                        ▼                        ▼
┌──────────────────┐       ┌──────────────────┐        ... more specialists
```

```
│  analyst_entry   │        │ forecaster_entry   │
└──────────────────┘        └────────────────────┘
         │                           │
         ▼                           ▼

┌──────────────────┐        ┌────────────────────┐
│     analyst      │        │    forecaster      │
│ Tools: execute_  │        │ Tools: execute_    │
│ python, list_    │        │ python, list_      │
│ output_files     │        │ output_files       │
└──────────────────┘        └────────────────────┘
    │      │                    │      │
    │      │                    │      │
    ▼      ▼                    ▼      ▼
 tools  manager_review      tools  manager_review
    │      │                    │      │
    └──►analyst                 └──►forecaster
    (loop) │                   (loop) │
           │ ACCEPT                   │ ACCEPT
           ▼                          ▼

┌──────────────────────────────────────────────────────────────────────┐
│                        code_reviewer_entry                           │
└──────────────────────────────────────────────────────────────────────┘
                                   │
                                   ▼

┌──────────────────────────────────────────────────────────────────────┐
│                           code_reviewer                              │
│  Tools: execute_python, list_output_files                            │
│  Job: Validate all outputs, fix errors, fill gaps                    │
└──────────────────────────────────────────────────────────────────────┘
                                   │
                                   ▼

┌──────────────────────────────────────────────────────────────────────┐
│                          manager_review                              │
└──────────────────────────────────────────────────────────────────────┘
                                   │
                                   │ ACCEPT
                                   ▼

┌──────────────────────────────────────────────────────────────────────┐
│                           reporter_entry                             │
└──────────────────────────────────────────────────────────────────────┘
                                   │
                                   ▼

┌──────────────────────────────────────────────────────────────────────┐
│                              reporter                                │
│  Tools: list_output_files, write_markdown_report                     │
│  Job: Write comprehensive Markdown report with all findings & charts │
└──────────────────────────────────────────────────────────────────────┘
                                   │
                                   ▼

┌──────────────────────────────────────────────────────────────────────┐
│                          manager_review                              │
│                            ACCEPT → END                              │
└──────────────────────────────────────────────────────────────────────┘
```

Rework Loop Detail

```
┌──────────────┐          ┌─────────┐          ┌──────────────────┐
│  specialist  │◄─────────│ tools   │          │  manager_review  │
│  (iteration  │          │         │          │                  │
```

```
│   1..5)      │──────►│ MCP    │         │  ACCEPT? ──────► next stage
│              │       │ calls  │         │                  │
│              │──────────────────────────►│  REWORK? ──────► specialist_entry
└──────────────┘                          │  (max 1 rework)  │
                                          └──────────────────┘


Safeguards:
- MAX_SPECIALIST_ITERS = 5 (per specialist, including rework iterations)
- MAX_REWORKS_PER_STAGE = 1 (manager can only send back once)
- data_loader and planner: auto-ACCEPT (no LLM review call)
```

---

6. State Management

`AnalysisState` (extends `MessagesState`)

```python
class AnalysisState(MessagesState):
    current_stage: str        # Which specialist is currently active
    stages_to_run: list[str]  # Dynamic pipeline set by planner
    stage_attempts: dict      # {stage_name: attempt_count} for rework tracking
```

Field	Set By	Used By
`messages`	All nodes (append-only)	All nodes — the conversation history
`current_stage`	`make_stage_entry()` nodes	`manager_router()` — knows which stage just finished
`stages_to_run`	`planner_node()`	`manager_router()` — determines next stage
`stage_attempts`	`manager_review_node()`	`manager_review_node()` — enforces rework limits

State Flow Example

```
Step 1: START
  current_stage: undefined
  stages_to_run: undefined
  stage_attempts: {}
```

```
Step 2: data_loader_entry
  current_stage: "data_loader"

Step 3: data_loader executes → calls query_mill_data tool → returns

Step 4: manager_review (auto-accept)
  stage_attempts: {"data_loader": 1}

Step 5: planner_entry
  current_stage: "planner"

Step 6: planner executes → "SPECIALISTS: analyst, forecaster"
  stages_to_run: ["data_loader", "planner", "analyst", "forecaster",
                  "code_reviewer", "reporter"]
  stage_attempts: {"data_loader": 1, "planner": 1}

Step 7: analyst_entry → analyst → tools (loop) → manager_review
  stage_attempts: {"data_loader": 1, "planner": 1, "analyst": 1}

Step 8: forecaster_entry → forecaster → tools (loop) → manager_review
  ...and so on until reporter → END
```

---

7. The Specialist Pool

Fixed Stages (Always Run)

Stage	Position	Purpose
`data_loader`	First	Load data from PostgreSQL into in-memory DataFrames
`planner`	Second	Analyze request, choose specialists
`code_reviewer`	Second-to-last	Validate all outputs, fix errors
`reporter`	Last	Write final Markdown report

Dynamic Specialists (Selected by Planner)

Specialist	Trigger Keywords	Analysis Capabilities
analyst	"analyze", "statistics", "overview", general/vague requests	Descriptive stats, distributions, SPC (Xbar, UCL/LCL), correlation heatmaps, Cp/Cpk, missing data, rolling means
forecaster	"predict", "forecast", "trend", "future", "seasonality"	Prophet forecasting, ARIMA/SARIMAX, seasonal decomposition, changepoint detection, shift-level patterns
anomaly_detective	"anomaly", "unusual", "root cause", "equipment problems"	Isolation Forest, DBSCAN clustering, SHAP root cause, rolling anomaly scores, operating regime detection
bayesian_analyst	"uncertainty", "confidence", "how sure", "compare regimes"	Bootstrap posteriors, credible intervals, Bayesian A/B testing, probabilistic Cpk, effect size estimation
optimizer	"optimize", "best settings", "setpoints", "tradeoffs"	Pareto frontiers, what-if simulation, optimal windows per ore type, sensitivity tornado charts, Monte Carlo risk
shift_reporter	"shift", "KPI", "energy", "benchmarking", "handover"	Per-shift KPIs, Mann-Whitney tests, mill ranking, energy kWh/ton, downtime analysis, handover reports

Planner Decision Rules

```
User Request                                        → Planner Selects
──────────────────────────────────────────────────────────────────────
"Analyze Mill 8 for the last 7 days"                → analyst, shift_reporter
"Predict PSI80 trends for Mill 8"                   → analyst, forecaster
"Find anomalies and explain root causes"            → anomaly_detective
"Optimize Mill 8 setpoints"                         → analyst, optimizer
"Full comprehensive report"                         → analyst, anomaly_detective,
shift_reporter
```

```
"How confident are we in the PSI80 range?"      → analyst, bayesian_analyst
"Compare shift performance with KPIs"           → shift_reporter
```

---

8. System Prompts — Deep Dive

Domain Context (Shared by ALL agents)

Every agent receives `DOMAIN_CONTEXT` — a shared knowledge base about the factory:

```
┌──────────────────────────────────────────────────────────────────────┐
│                        DOMAIN_CONTEXT                                │
│                                                                      │
│  Factory: Ore dressing plant with 12 ball mills                      │
│  Database: MILL_XX tables (minute-level time-series)                 │
│                                                                      │
│  Columns:                                                            │
│  ┌──────────────┬──────────────────────────────────────────────┐    │
│  │ TimeStamp    │ Index column, minute resolution              │    │
│  │ Ore          │ Feed rate (t/h)                              │    │
│  │ WaterMill    │ Water to mill (m³/h)                         │    │
│  │ WaterZumpf   │ Water to sump (m³/h)                         │    │
│  │ Power        │ Mill motor power (kW)                        │    │
│  │ ZumpfLevel   │ Sump level                                   │    │
│  │ PressureHC   │ Hydrocyclone pressure                        │    │
│  │ DensityHC    │ Hydrocyclone density                         │    │
│  │ FE           │ Iron content                                 │    │
│  │ PulpHC       │ Hydrocyclone pulp                            │    │
│  │ PumpRPM      │ Pump speed                                   │    │
│  │ MotorAmp     │ Mill motor current (A)                       │    │
│  │ PSI80        │ Product fineness 80% passing (µm) — TARGET   │    │
│  │ PSI200       │ Product fineness 200 mesh (%) — TARGET       │    │
│  └──────────────┴──────────────────────────────────────────────┘    │
│                                                                      │
│  Ore Quality (lab): Shisti, Daiki, Grano, Class_12, Class_15         │
│                                                                      │
│  Process Relationships:                                              │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  MVs (control) ──► CVs (respond) ──► Targets (quality)       │   │
│  │                                                              │   │
│  │  Ore, WaterMill ──► PressureHC   ──► PSI80 (65-85 µm)        │   │
│  │  WaterZumpf,    ──► DensityHC    ──► PSI200 (55-75%)         │   │
│  │  MotorAmp       ──► PulpHC                                   │   │
│  │                                                              │   │
│  │  DVs (uncontrollable): Shisti, Daiki, Grano, Class_12/15     │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  Shifts: S1 (06-14), S2 (14-22), S3 (22-06)                          │
```

```
│  Energy metric: Power / Ore = kWh/ton                                  │
└────────────────────────────────────────────────────────────────────────┘
```

Prompt Structure Per Agent

```
┌──────────────────────────────────────────────────────────────────────┐
│                    SPECIALIST PROMPT STRUCTURE                       │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  DOMAIN_CONTEXT (shared)                                     │   │
│  │  - Factory description, table structure, column meanings     │   │
│  │  - Process relationships (MV → CV → Target)                  │   │
│  │  - Shift definitions, energy metrics                         │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                              +                                       │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  ROLE DEFINITION                                             │   │
│  │  - "You are the [Role Name]"                                 │   │
│  │  - Specific responsibilities                                 │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                              +                                       │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  DATA ACCESS INSTRUCTIONS                                    │   │
│  │  - get_df('mill_data_8'), list_dfs()                         │   │
│  │  - How to access loaded DataFrames                           │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                              +                                       │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  ANALYSIS STEPS (numbered, with code examples)               │   │
│  │  - Step 1: Do this analysis                                  │   │
│  │  - Step 2: Create this chart                                 │   │
│  │  - Step 3: Print these statistics                            │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                              +                                       │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  CRITICAL RULES                                              │   │
│  │  - Chart quality rules (sizes, labels, save paths)           │   │
│  │  - Data handling rules (NaN, resampling)                     │   │
│  │  - Output rules (print numbers, save charts)                 │   │
│  └──────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────┘
```

Data Loader Prompt — Key Details

- **ONLY job**: Call `query_mill_data` or `query_combined_data`
- **CRITICAL**: Must compute `start_date` and `end_date` for SQL filtering
- Injects `{TODAY_DATE}` at runtime for date calculations

- Uses `query_combined_data` when ore quality (Shisti, Daiki) is mentioned
- Must NOT analyze data — only load and summarize

Planner Prompt — Key Details

- Outputs **exact format**: `SPECIALISTS: agent1, agent2\nRATIONALE: ...`
- Max 1-4 specialists, never all 6
- Order matters: foundational analysis first (analyst), then specialized
- Has fallback rules for common request patterns

Manager Review Prompt — Key Details

- Evaluates: completeness, quality, correctness, actionability
- Outputs exactly `ACCEPT: [reason]` or `REWORK: [instructions]`
- Auto-accepts `data_loader` and `planner` without LLM call

---

9. Graph Builder — `build_graph()`

Function Signature

```
def build_graph(
    tools: list[BaseTool],
    api_key: str,
    on_progress: Optional[Callable[[str, str], None]] = None,
) -> StateGraph:
```

- **Input**: LangChain tools (from MCP server via `client.py`) + Google API key + optional progress callback
- **Output**: Compiled LangGraph state machine ready for `.ainvoke()`
- `on_progress`: When provided, every node calls `on_progress(stage, message)` to report real-time progress. If `None`, a no-op lambda is used instead. See Section 10: Progress Reporting System.

Build Process (Step by Step)

```
build_graph(tools, api_key, on_progress=callback)
    │
    ├── 1. Initialize progress callback
    │      _progress = on_progress or (lambda stage, msg: None)  # no-op fallback
    │
    ├── 2. Create LLM instance
    │      llm = ChatGoogleGenerativeAI(model="gemini-3.1-flash-lite-preview")
    │
    ├── 3. Organize tools by name
    │      tools_by_name = {t.name: t for t in tools}
    │
    ├── 4. Define tool sets per specialist
    │      TOOL_SETS = {
    │        "data_loader":       ["query_mill_data", "query_combined_data",
"get_db_schema"],
    │        "analyst":           ["execute_python", "list_output_files"],
    │        "forecaster":        ["execute_python", "list_output_files"],
    │        "anomaly_detective": ["execute_python", "list_output_files"],
    │        "bayesian_analyst":  ["execute_python", "list_output_files"],
    │        "optimizer":         ["execute_python", "list_output_files"],
    │        "shift_reporter":    ["execute_python", "list_output_files"],
    │        "code_reviewer":     ["execute_python", "list_output_files"],
    │        "reporter":          ["list_output_files", "write_markdown_report"],
    │      }
    │
    ├── 5. Bind tools to LLM per specialist
    │      specialist_llms[name] = llm.bind_tools(stage_tools)
    │
    ├── 6. Create helper functions
    │      ├── truncate()           — trim text to character limit
    │      ├── normalize_content()  — handle list/string content
    │      ├── compress_messages()  — sliding window + truncation
    │      ├── strip_tool_messages()— convert ToolMessage → AIMessage
    │      └── build_focused_context() — specialist-specific context
    │
    ├── 7. Create node functions (all nodes use _progress callback)
    │      ├── make_specialist_node(name) — factory for specialist nodes
    │      │     calls _progress on: start, tool calls, completion
    │      ├── planner_node()             — parses SPECIALISTS response
    │      │     calls _progress on: start, pipeline selection
    │      ├── manager_review_node()      — ACCEPT/REWORK decision
    │      │     calls _progress on: review start, accept/rework decision
    │      ├── tool_node()                — executes MCP tool calls
    │      │     calls _progress on: each tool execution
    │      └── make_stage_entry(name)     — sets current_stage
    │
    ├── 8. Assemble the graph
    │      graph = StateGraph(AnalysisState)
    │      │
    │      ├── Register ALL entry + specialist nodes (9 stages × 2)
    │      ├── Register planner node
    │      ├── Register tools node
    │      ├── Register manager_review node
    │      │
    │      ├── Set entry point: data_loader_entry
    │      │
```

```
    │       ├── Wire: entry → specialist (for all 9 stages)
    │       ├── Wire: planner → manager_review
    │       ├── Wire: specialist → tools OR manager_review
    │       ├── Wire: tools → back to calling specialist
    │       └── Wire: manager_review → next stage OR rework OR END
    │
    └── 9. Return graph.compile()
```

---

10. Progress Reporting System

`build_graph()` accepts an optional `on_progress` callback that enables **real-time visibility** into what the agents are doing. When the graph is invoked from `api_endpoint.py`, this callback appends `ProgressMessage` entries to the in-memory tracking dictionary, which the frontend picks up on each poll.

Stage Labels

Human-readable labels are defined in `_STAGE_LABELS` and used in all progress messages:

```python
_STAGE_LABELS: dict[str, str] = {
    "data_loader":       "Data Loader",
    "planner":           "Planner",
    "analyst":           "Analyst",
    "forecaster":        "Forecaster",
    "anomaly_detective": "Anomaly Detective",
    "bayesian_analyst":  "Bayesian Analyst",
    "optimizer":         "Optimizer",
    "shift_reporter":    "Shift Reporter",
    "code_reviewer":     "Code Reviewer",
    "reporter":          "Reporter",
    "manager":           "Manager",
}
```

The helper `_label(stage)` returns the human-readable name for a stage key.

Progress Messages by Node Type

Every node in the graph calls `_progress(stage, message)`. Here's exactly what each node reports:

Specialist Nodes (`make_specialist_node`)

```
┌──────────────────────────────────────────────────────────────────┐
│  specialist_node(state)                                          │
│                                                                  │
│  Entry:                                                          │
│    _progress("analyst", "Analyst working (step 2/5)...")         │
│                                                                  │
│  If iteration cap reached:                                       │
│    _progress("analyst", "Analyst finished (iteration cap).")     │
│                                                                  │
│  If tool calls in response:                                      │
│    _progress("analyst", "Analyst calling tools: execute_python") │
│                                                                  │
│  If no tool calls (done):                                        │
│    _progress("analyst", "Analyst completed.")                    │
└──────────────────────────────────────────────────────────────────┘
```

Planner Node

```
┌──────────────────────────────────────────────────────────────────┐
│  planner_node(state)                                             │
│                                                                  │
│  Entry:                                                          │
│    _progress("planner", "Planning analysis — selecting           │
│               specialists...")                                    │
│                                                                  │
│  After selection:                                                │
│    _progress("planner", "Pipeline: Analyst → Forecaster →        │
│               Shift Reporter")                                   │
│    (uses human-readable labels from _STAGE_LABELS)               │
└──────────────────────────────────────────────────────────────────┘
```

Manager Review Node

```
┌──────────────────────────────────────────────────────────────┐
│  manager_review_node(state)                                  │
│                                                              │
│  Entry:                                                      │
│    _progress("manager", "Reviewing Analyst output...")       │
│                                                              │
│  ACCEPT decision:                                            │
│    _progress("manager", "Analyst — accepted.")               │
│                                                              │
│  REWORK decision:                                            │
│    _progress("manager", "Analyst — rework requested.")       │
└──────────────────────────────────────────────────────────────┘
```

Tool Node

```
┌──────────────────────────────────────────────────────────────┐
│  tool_node(state)                                            │
│                                                              │
│  For each tool call:                                         │
│    _progress("tools", "Executing tool: execute_python")      │
│    _progress("tools", "Executing tool: query_mill_data")     │
└──────────────────────────────────────────────────────────────┘
```

Manager Router (Stage Transitions)

```
┌──────────────────────────────────────────────────────────────┐
│  manager_router(state)                                       │
│                                                              │
│  On ACCEPT → advance:                                        │
│    _progress("system", "Advancing: Analyst → Forecaster")    │
│    (uses human-readable labels from _STAGE_LABELS)           │
└──────────────────────────────────────────────────────────────┘
```

Progress Timeline Example

For a request like *"Analyze Mill 8 for the last 7 days"* with planner selecting `analyst` + `shift_reporter`:

```
t=0s    [system]       "Connecting to MCP server..."
t=1s    [system]       "Building agent pipeline..."
t=2s    [data_loader]  "Data Loader working (step 1/5)..."
t=3s    [data_loader]  "Data Loader calling tools: query_mill_data"
```

```
t=5s    [tools]        "Executing tool: query_mill_data"
t=8s    [data_loader]  "Data Loader completed."
t=9s    [manager]      "Reviewing Data Loader output..."
t=9s    [manager]      "Data Loader — accepted."
t=10s   [system]       "Advancing: Data Loader → Planner"
t=11s   [planner]      "Planning analysis — selecting specialists..."
t=13s   [planner]      "Pipeline: Analyst → Shift Reporter"
t=14s   [manager]      "Reviewing Planner output..."
t=14s   [manager]      "Planner — accepted."
t=15s   [system]       "Advancing: Planner → Analyst"
t=16s   [analyst]      "Analyst working (step 1/5)..."
t=17s   [analyst]      "Analyst calling tools: execute_python"
t=18s   [tools]        "Executing tool: execute_python"
t=25s   [analyst]      "Analyst working (step 2/5)..."
t=26s   [analyst]      "Analyst calling tools: execute_python"
t=27s   [tools]        "Executing tool: execute_python"
t=35s   [analyst]      "Analyst completed."
t=36s   [manager]      "Reviewing Analyst output..."
t=38s   [manager]      "Analyst — accepted."
t=39s   [system]       "Advancing: Analyst → Shift Reporter"
...     (shift_reporter, code_reviewer, reporter follow)
t=180s  [system]       "Analysis complete."
```

The frontend displays these messages in a `ProgressFeed` component as a scrollable list with stage icons and timestamps. See api_endpoint.md for details on how the callback is created and wired.

---

11. Routing Logic — How Decisions Are Made

Three Router Functions

```
┌──────────────────────────────────────────────────────────────────────┐
│                      specialist_router(state)                        │
│                                                                      │
│  Called AFTER a specialist node runs.                                │
│                                                                      │
│  if last_message.tool_calls:                                         │
│      return "tools"           ──► tool_node executes the calls       │
│  else:                                                               │
│      return "manager_review"  ──► manager evaluates the output       │
└──────────────────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────────────────┐
│                        after_tools(state)                            │
│                                                                      │
```

```
│  Called AFTER tools execute. Routes back to the calling specialist.   │
│                                                                        │
│  Walk backwards through messages, find the last AIMessage with        │
│  tool_calls and a name → return that name (e.g., "analyst")          │
│  This sends tool results back to the specialist for next iteration.  │
└────────────────────────────────────────────────────────────────────────┘


┌────────────────────────────────────────────────────────────────────────┐
│                      manager_router(state)                             │
│                                                                        │
│  Called AFTER manager_review. Three possible outcomes:                 │
│                                                                        │
│  1. REWORK → return "{current}_entry"                                  │
│     Manager found issues, sends specialist back for another try        │
│                                                                        │
│  2. ACCEPT → return "{next_stage}_entry"                               │
│     Look up current stage in stages_to_run, advance to idx + 1        │
│                                                                        │
│  3. Pipeline complete → return "end" → END                             │
│     No more stages in stages_to_run                                    │
└────────────────────────────────────────────────────────────────────────┘
```

Routing Diagram

```
                    specialist
                   ┌──────────┐
                   │          │
          ┌──yes──┤tool_calls?├──no──┐
          │        │          │       │
          ▼        └──────────┘       ▼
     ┌─────────┐                ┌──────────────┐
     │  tools  │                │manager_review│
     └────┬────┘                └──────┬───────┘
          │                            │
          ▼                      ┌─────┴─────┐
   after_tools()                 │     │     │
          │              REWORK│ ACCEPT  pipeline
          │                    │     │   complete
          ▼                    ▼     ▼     ▼
    back to same        current  next   END
    specialist          _entry   stage
    (next iteration)            _entry
```

---

12. Tool Binding — Per-Specialist Tools

```
┌──────────────────────┬──────────────────────────────────────────────────┐
│    Specialist        │  Bound Tools                                     │
├──────────────────────┼──────────────────────────────────────────────────┤
│  data_loader         │  query_mill_data, query_combined_data,           │
│                      │  get_db_schema                                   │
├──────────────────────┼──────────────────────────────────────────────────┤
│  analyst             │  execute_python, list_output_files               │
│  forecaster          │  execute_python, list_output_files               │
│  anomaly_detective   │  execute_python, list_output_files               │
│  bayesian_analyst    │  execute_python, list_output_files               │
│  optimizer           │  execute_python, list_output_files               │
│  shift_reporter      │  execute_python, list_output_files               │
│  code_reviewer       │  execute_python, list_output_files               │
├──────────────────────┼──────────────────────────────────────────────────┤
│  reporter            │  list_output_files, write_markdown_report        │
├──────────────────────┼──────────────────────────────────────────────────┤
│  planner             │  NONE (text-only LLM call, no tools)             │
│  manager             │  NONE (text-only LLM call, no tools)             │
└──────────────────────┴──────────────────────────────────────────────────┘
```

Why Tool Isolation?

- `data_loader` can **only** load data — can't accidentally run analysis code
- `reporter` can **only** list files and write reports — can't run arbitrary Python
- All 6 analysis specialists share `execute_python` — they write code that runs in a sandboxed namespace with scientific libraries pre-loaded

---

13. Context Management — Token Budget

Constants

```
MAX_TOOL_OUTPUT_CHARS = 2000   # Truncate tool outputs to 2KB
MAX_AI_MSG_CHARS      = 3000   # Truncate AI messages to 3KB
MAX_MESSAGES_WINDOW   = 14     # Keep last 14 messages in context
MAX_SPECIALIST_ITERS = 5      # Max iterations per specialist
```

Context Building Strategy

```
┌──────────────────────────────────────────────────────────────────────┐
│              build_focused_context(messages, stage_name)             │
│                                                                      │
│  Purpose: Build a FOCUSED message list for a specific specialist     │
│  instead of sending the entire conversation history.                 │
│                                                                      │
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │  1. Extract the original user question                       │   │
│   │     → Always included as first message                       │   │
│   └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │  2. Summarize prior stages compactly                         │   │
│   │     → Data loading results: truncated to 200 chars           │   │
│   │     → Python outputs: truncated to 1200 chars                │   │
│   │     → Other specialist findings: truncated to 400 chars      │   │
│   │     → Keep last 8 summaries only                             │   │
│   │     → Wrapped as "[Prior analysis context]"                  │   │
│   └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │  3. Include current specialist's OWN messages + tools        │   │
│   │     → Full history for this specialist                       │   │
│   │     → Tool results matched by tool_call_id                   │   │
│   │     → Manager REWORK instructions if relevant                │   │
│   └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │  4. Apply compression to current stage messages              │   │
│   │     → Sliding window: keep first + last 14 messages          │   │
│   │     → Truncate ToolMessages to 2KB                           │   │
│   │     → Truncate AIMessages to 3KB                             │   │
│   └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  Result: [user_question, prior_summary, specialist_own_msgs...]      │
└──────────────────────────────────────────────────────────────────────┘
```

Why Focused Context?

Without it, by the time the 4th specialist runs, the message history could be **50+ messages** with large tool outputs. This would exceed the LLM's context window. The focused context builder ensures each specialist sees:

1. The original question (always relevant)
2. A **compact summary** of what prior stages found
3. Its **own full history** (for tool call continuation and rework)

---

14. Manager Review & Rework Loop

```
┌──────────────────────────────────────────────────────────────────────┐
│                    manager_review_node(state)                        │
│                                                                      │
│  Input: current_stage, stage_attempts from state                     │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  IF current == "data_loader" or "planner":                   │   │
│  │     → Auto-ACCEPT (no LLM call needed)                       │   │
│  │     → These stages are deterministic, no review needed       │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  IF attempts >= MAX_REWORKS_PER_STAGE (1):                   │   │
│  │     → Auto-ACCEPT (max reworks reached)                      │   │
│  │     → Prevents infinite rework loops                         │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  OTHERWISE:                                                  │   │
│  │     → Call LLM with MANAGER_REVIEW_PROMPT                    │   │
│  │     → LLM evaluates: completeness, quality, correctness      │   │
│  │     → Outputs "ACCEPT: [reason]" or "REWORK: [instructions]"│   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  REWORK flow:                                                │   │
│  │     → manager_router returns "{current}_entry"               │   │
│  │     → Stage entry resets current_stage                       │   │
│  │     → Specialist runs again with REWORK instructions         │   │
│  │       visible in its focused context                          │   │
│  └──────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────┘
```

---

15. File Dependencies Map

```
graph_v3.py
    │
    ├── imports from ──────────────────────────────────────────────────
    │   ├── datetime (datetime)
    │   ├── typing (Callable, Optional)
    │   ├── langchain_core.messages (SystemMessage, HumanMessage, AIMessage,
    │   │                            ToolMessage, BaseMessage)
    │   ├── langchain_core.tools (BaseTool)
    │   ├── langchain_google_genai (ChatGoogleGenerativeAI)
    │   └── langgraph.graph (StateGraph, MessagesState, END, START)
    │
    ├── called by ─────────────────────────────────────────────────────
    │   ├── api_endpoint.py
    │   │     _run_analysis_background() →
```

```
│   │       build_graph(tools, api_key, on_progress=callback)
│   │     Then: graph.ainvoke({"messages": [HumanMessage(prompt)]})
│   │     (on_progress callback created by _make_progress_callback)
│   │
│   └── main.py
│         main() → build_graph(tools, api_key)
│         Demo CLI entry point for testing (no progress callback)
│
├── receives tools from ──────────────────────────────────────────
│   └── client.py
│         get_mcp_tools(session) → list[BaseTool]
│         Converts MCP tool descriptors to LangChain StructuredTools
│         │
│         └── connects to ── server.py (MCP Server, port 8003)
│                            │
│                            └── tools/__init__.py (registry)
│                                 ├── db_tools.py
│                                 │     ├── get_db_schema
│                                 │     ├── query_mill_data → PostgreSQL
│                                 │     └── query_combined_data → PostgreSQL
│                                 ├── python_executor.py
│                                 │     └── execute_python
│                                 │           (pandas, numpy, matplotlib,
│                                 │            Prophet, sklearn, shap,
│                                 │            statsmodels, hmmlearn)
│                                 ├── report_tools.py
│                                 │     ├── list_output_files
│                                 │     └── write_markdown_report
│                                 ├── session_tools.py
│                                 │     └── set_output_directory
│                                 └── output_dir.py
│                                       └── shared mutable OUTPUT_DIR state
│
├── writes to ────────────────────────────────────────────────────
│   └── output/{analysis_id}/
│         ├── *.png  (charts from execute_python → plt.savefig)
│         └── *.md   (reports from write_markdown_report)
│
└── reads from ───────────────────────────────────────────────────
    └── .env (GOOGLE_API_KEY — loaded by api_endpoint.py or main.py)
```

---

16. MCP Tools Reference

Tool Registry (7 tools)

#	Tool Name	File	Purpose	Used By
1	`get_db_schema`	db_tools.py	Inspect PostgreSQL	data_loader

#	Tool Name	File	Purpose	Used By
			table/column metadata	
2	`query_mill_data`	db_tools.py	Load MILL_XX sensor data → in-memory DataFrame	data_loader
3	`query_combined_data`	db_tools.py	Load mill + ore_quality joined data	data_loader
4	`execute_python`	python_executor.py	Run Python code with scientific libraries	All 6 specialists + code_reviewer
5	`list_output_files`	report_tools.py	List generated files in output directory	All 6 specialists + code_reviewer + reporter
6	`write_markdown_report`	report_tools.py	Write final Markdown report to output dir	reporter
7	`set_output_directory`	session_tools.py	Set per-analysis output subfolder	Called by api_endpoint.py before graph runs

execute_python Namespace

The Python execution environment pre-loads these into the `exec()` namespace:

```
execute_python namespace

Core:
  df           — default loaded DataFrame
  get_df(name) — get any named DataFrame from store
  list_dfs()   — list all loaded DataFrames with shapes
  pd, np, plt, sns, scipy_stats, os, json
  OUTPUT_DIR   — per-analysis output path

Advanced (graceful fallback if not installed):
  Prophet         — Facebook Prophet (time series)
  sm              — statsmodels.api
  tsa             — statsmodels.tsa.api
  pmdarima        — auto_arima
  IsolationForest — sklearn.ensemble
  DBSCAN          — sklearn.cluster
  StandardScaler  — sklearn.preprocessing
  LinearRegression— sklearn.linear_model
  shap            — SHAP explainability
  hmm             — hmmlearn.hmm (Hidden Markov Models)
```

In-Memory DataFrame Store

```
_dataframes: dict[str, pd.DataFrame]

Shared across all tools in the MCP server process.
Written by: query_mill_data, query_combined_data
Read by: execute_python (via get_df() and df alias)

Example after loading:
{
  "mill_data_8": DataFrame(43200 rows × 14 cols),
  "mill_data_6": DataFrame(43200 rows × 14 cols),
  "combined_data": DataFrame(43200 rows × 19 cols),
}
```

---

17. Execution Examples

Example 1: "Analyze Mill 8 for the last 7 days"

```
Planner output: SPECIALISTS: analyst, shift_reporter
Pipeline: data_loader → planner → analyst → shift_reporter → code_reviewer →
reporter

┌─ data_loader ──────────────────────────────────────────────────────────┐
│  Calls: query_mill_data(mill_number=8, start_date="2026-03-25",        │
│                          end_date="2026-04-01")                         │
│  Result: "mill_data_8" → 10,080 rows loaded                             │
└────────────────────────────────────────────────────────────────────────┘

        ▼ manager: auto-ACCEPT
┌─ planner ──────────────────────────────────────────────────────────────┐
│  "SPECIALISTS: analyst, shift_reporter"                                 │
│  "RATIONALE: User wants general analysis + shift comparison"            │
└────────────────────────────────────────────────────────────────────────┘

        ▼ manager: auto-ACCEPT
┌─ analyst (iterations 1-3) ─────────────────────────────────────────────┐
│  Iter 1: execute_python → descriptive stats, distributions              │
│  Iter 2: execute_python → SPC control chart, correlation heatmap        │
│  Iter 3: execute_python → process capability Cp/Cpk                     │
│  Output: 5 charts saved, statistics printed                             │
└────────────────────────────────────────────────────────────────────────┘

        ▼ manager: ACCEPT (charts + numbers look good)
┌─ shift_reporter (iterations 1-2) ──────────────────────────────────────┐
│  Iter 1: execute_python → shift KPIs, energy analysis                   │
│  Iter 2: execute_python → mill ranking, handover summary                │
│  Output: 3 charts saved, shift comparison table printed                 │
└────────────────────────────────────────────────────────────────────────┘

        ▼ manager: ACCEPT
┌─ code_reviewer ────────────────────────────────────────────────────────┐
│  Calls: list_output_files → 8 PNG files found                           │
│  Reviews stdout for errors → none found                                 │
│  Output: "All outputs validated. 8 charts generated."                   │
└────────────────────────────────────────────────────────────────────────┘

        ▼ manager: ACCEPT
┌─ reporter ─────────────────────────────────────────────────────────────┐
│  Calls: list_output_files → gets exact filenames                        │
│  Reads all previous messages for statistics                             │
│  Calls: write_markdown_report("mill_8_analysis.md", content)            │
│  Output: 2000+ word professional report with embedded charts            │
└────────────────────────────────────────────────────────────────────────┘

        ▼ manager: ACCEPT → END
```

Example 2: "Predict PSI80 trends for Mill 8"

```
Planner output: SPECIALISTS: analyst, forecaster
Pipeline: data_loader → planner → analyst → forecaster → code_reviewer → reporter

  data_loader: Loads mill_data_8
  planner: Selects analyst + forecaster
  analyst: Descriptive stats, time series overview
  forecaster: Prophet forecast (8h ahead), seasonal decomposition,
```

```
                changepoint detection, confidence intervals
  code_reviewer: Validates forecast outputs
  reporter: Report with forecast charts and prediction summary
```

Example 3: "Find anomalies in Mill 8 and explain root causes"

```
Planner output: SPECIALISTS: anomaly_detective
Pipeline: data_loader → planner → anomaly_detective → code_reviewer → reporter

  data_loader: Loads mill_data_8
  planner: Selects anomaly_detective only
  anomaly_detective: Isolation Forest → anomaly timeline →
                     SHAP root cause → DBSCAN regimes →
                     top 5 anomaly events with timestamps
  code_reviewer: Validates anomaly outputs
  reporter: Report with anomaly charts and root cause explanations
```

---

18. Configuration Constants

```python
# LLM Model
GEMINI_MODEL = "gemini-3.1-flash-lite-preview"

# Token-budget controls
MAX_TOOL_OUTPUT_CHARS = 2000   # Max characters per tool output in context
MAX_AI_MSG_CHARS      = 3000   # Max characters per AI message in context
MAX_MESSAGES_WINDOW   = 14     # Sliding window size for message compression
MAX_SPECIALIST_ITERS  = 5      # Max tool-call iterations per specialist

# Pipeline structure
FIXED_PREFIX = ["data_loader", "planner"]          # Always first
FIXED_SUFFIX = ["code_reviewer", "reporter"]       # Always last
SPECIALIST_POOL = [                                 # Planner chooses from these
    "analyst", "forecaster", "anomaly_detective",
    "bayesian_analyst", "optimizer", "shift_reporter",
]

# Quality gates
MAX_REWORKS_PER_STAGE = 1   # Manager can send back at most once

# External services
MCP_SERVER_URL = "http://localhost:8003/mcp"  # MCP Server (server.py)
GOOGLE_API_KEY = from .env                     # Gemini API key
```

Quick Reference: Entry Points

Entry Point	File	Purpose
CLI	`main.py`	Run demo analyses from command line
REST API	`api_endpoint.py`	`POST /api/v1/agentic/analyze` → background graph execution
Frontend	`/ai-chat/page.tsx`	User-facing chat UI → polls API for results

All three call `build_graph(tools, api_key)` from `graph_v3.py` and then `graph.ainvoke()`.